Guía Evaluativa segundo momento:
Automatización de Flujos de Datos con Python

De Excel (openpyxl) a Consola (tabulate) e Informe Ejecutivo (ReportLab)

1. Introducción y Contexto

En el entorno profesional actual, la gestión manual de información guardada en hojas de cálculo consume tiempo valioso y está expuesta a errores humanos. La automatización mediante lenguaje Python permite leer bases de datos, procesarlas en memoria, presentarlas de forma limpia en consola y compilar reportes ejecutivos listos para impresión.

El Caso de Negocio: La empresa TecnoVentas cuenta con una base de datos centralizada en Excel llamada **ventas_tecnologia_v2.xlsx**, la cual contiene un inventario de productos tecnológicos con su ID, descripción, categoría, precio unitario y unidades vendidas. Tu objetivo como desarrollador Python es construir un script que lea la hoja de cálculo, organice la información en memoria, la visualice formateada en la consola interactiva mediante la librería **tabulate** y exporte automáticamente un reporte profesional en PDF con la librería **ReportLab**.

2. Objetivos de Aprendizaje

- **Dominar la carga de libros de cálculo (.xlsx)** existentes con openpyxl.load_workbook() y comprender la diferencia entre fórmulas y resultados numéricos mediante data_only=True.
- **Extraer rangos dinámicos de datos** utilizando iter_rows() sin depender de límites rígidos de filas o columnas.
- **Formatear y alinear tablas en consola** mediante la librería tabulate para mejorar la legibilidad de textos y números.
- **Comprender la arquitectura de ReportLab** para la creación de PDFs profesionales (modelo de elementos fluibles story, Paragraph, Table, TableStyle y colWidths).
- **Desarrollar habilidades de resolución de problemas** mediante la implementación de filtros, cálculo de métricas y desarrollo de un menú interactivo en consola.

3. Preparación del Entorno

Antes de iniciar el desarrollo, debes instalar las tres librerías principales utilizadas en este taller. Abre tu terminal de comandos o consola e ingresa la siguiente instrucción:

```
# Comandos de instalación en terminal/consola
pip install openpyxl tabulate reportlab
```

4. Tutorial Guiado Paso a Paso

Paso 1: Cargar el Archivo de Excel (load_workbook)

Para trabajar con un archivo de Excel existente, utilizamos la función **load_workbook** del módulo **openpyxl**. Observa y escribe la siguiente estructura de código:

```
import openpyxl

# 1. Cargar el libro de trabajo existente
wb = openpyxl.load_workbook("ventas_tecnologia_v2.xlsx", data_only=True)

# 2. Seleccionar la hoja activa (primera pestaña)
hoja = wb.active

print(f"Hoja cargada exitosamente: {hoja.title}")
```

Explicación Teórica Detallada:

- **openpyxl.load_workbook('...'):** Carga la estructura binaria del archivo .xlsx en la memoria RAM y retorna un objeto de tipo Workbook.
- **El parámetro data_only=True:** Es un parámetro fundamental. Si una celda contiene una fórmula de Excel (ejemplo =C2*D2), por defecto openpyxl leería el texto literal de la fórmula ('=C2*D2'). Al activar data_only=True, Python extrae únicamente el resultado numérico evaluado que Excel calculó y guardó la última vez.
- **Ubicación del archivo:** Al escribir solo el nombre del archivo 'ventas_tecnologia_v2.xlsx', Python asume que el archivo está guardado en la misma carpeta que tu script .py. De lo contrario, deberás indicar la ruta completa de la ubicación.

Paso 2: Lectura Dinámica de Registros (iter_rows)

Para leer tablas completas de forma eficiente, empleamos el método **iter_rows()**. Escribe la siguiente sección de código en tu entorno:

```python
# Crear una lista vacía para almacenar las filas
datos = []

# Recorrer desde la fila 2 (omitimos encabezados de la fila 1)
for fila in hoja.iter_rows(min_row=2, values_only=True):
    datos.append(list(fila))

print(f"Total de registros leídos en memoria: {len(datos)}")
```

Explicación Teórica Detallada:

- **min_row=2:** Le indica a openpyxl que inicie la lectura en la fila 2, omitiendo los encabezados de la fila 1 (ID, Producto, Categoría, Precio, Ventas).
- **Omisión de max_row:** Al no especificar max_row, el bucle se vuelve dinámico y lee automáticamente hasta la última fila que contenga datos. Si la tabla cambia de 22 a 500 filas, el código funcionará sin modificaciones.
- **values_only=True:** Indica que deseamos recibir directamente los valores contenidos en las celdas (números o texto), en lugar de recibir los objetos de celda de openpyxl (<Cell 'Ventas'.A2>).
- **list(fila):** Convierte la tupla inmutable devuelta por cada iteración en una lista manipulable y la almacena en la matriz principal datos.

Paso 3: Visualización Profesional en Consola (tabulate)

Para presentar los datos leídos de forma ordenada en pantalla, utilizamos la librería **tabulate**. Transcribe el código a continuación:

```python
from tabulate import tabulate

# 1. Nombres de columnas
encabezados = ["ID", "Producto", "Categoría", "Precio ($)", "Ventas (Unid)"]

# 2. Alineación por columna ("left", "center", "right")
alineacion = ("center", "left", "center", "right", "right")

# 3. Mostrar tabla formateada en consola
print("\n=== REPORTE DE VENTAS EN CONSOLA (22 PRODUCTOS) ===")
print(tabulate(datos, headers=encabezados, tablefmt="fancy_grid", colalign=alineacion))
```

Explicación Teórica Detallada:

- **headers=encabezados:** Suministra la lista de nombres para la fila superior de la tabla.
- **tablefmt='fancy_grid':** Aplica un marco visual con bordes dobles y esquinas bien definidas en la consola.
- **colalign=alineacion:** Define la alineación por columna. Por convención visual: identificadores y categorías se alinean al centro ('center'), descripciones a la izquierda ('left'), y valores numéricos a la derecha ('right') para alinear unidades, decenas y centenas verticalmente.

Paso 4: Generación del Informe Ejecutivo en PDF (ReportLab)

La librería **ReportLab** nos permite generar documentos PDF desde cero. Como es una herramienta avanzada, analizaremos su funcionamiento mediante 6 conceptos fundamentales:

1. La analogía del vagón de carga (Lista story): En ReportLab los documentos se construyen apilando bloques verticalmente. La lista story = [] actúa como un carrito de compras. Agregamos elementos con story.append() (títulos, párrafos, tablas, espacios) y se irán dibujando en la hoja de arriba hacia abajo.

2. El objeto Paragraph vs. Texto simple: Nunca debemos colocar cadenas de texto simples dentro de una tabla de PDF. El objeto Paragraph envuelve el texto y le otorga tres súper poderes: ajuste automático de línea (Word Wrap) para evitar que el texto se salga de la hoja, aplicación de estilos (tamaño, fuente, color) y soporte para etiquetas HTML como negrita e cursiva.

3. Conversión de la matriz de datos a Table: Para construir la tabla en el PDF, tomamos los datos leídos de Excel y convertimos cada celda individual en un objeto Paragraph. Luego unimos la lista de encabezados con la lista de datos ([encabezados] + filas) y se la pasamos al constructor Table().

4. Coordenadas de TableStyle (columna, fila): Para dar estilo visual (colores de fondo, cuadrícula, fuentes), usamos TableStyle. Las coordenadas funcionan como pares (columna, fila). La esquina superior izquierda es (0, 0), mientras que (-1, -1) representa la última celda disponible en la esquina inferior derecha.

5. Medidas en puntos tipográficos y colWidths: En ReportLab las dimensiones se miden en puntos (72 puntos = 1 pulgada). Una hoja tamaño Carta mide 612 puntos de ancho. Restando márgenes, disponemos de ~468 puntos de espacio útil. La propiedad colWidths = [40, 160, 100, 80, 80] suma 460 puntos, asegurando que la tabla ocupe todo el ancho sin desbordarse.

6. El botón de compilación (doc.build): La orden doc.build(story) procesa todos los elementos acomodados en el carrito story, calcula los saltos de página necesarios y compila el archivo .pdf final en el disco.

Revisa la estructura de código que implementa la lógica de ReportLab:

```python
from reportlab.lib.pagesizes import letter
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib import colors

# 1. Crear documento base
doc = SimpleDocTemplate("reporte_ejecutivo_ventas.pdf", pagesize=letter)
estilos = getSampleStyleSheet()

# 2. Estilos personalizados
estilo_titulo = ParagraphStyle(
    'EstiloTitulo', parent=estilos['Title'],
    fontName='Helvetica-Bold', fontSize=18,
    textColor=colors.HexColor("#1A365D"), spaceAfter=10
)
estilo_cuerpo = ParagraphStyle('EstiloCuerpo', parent=estilos['Normal'], fontSize=9)
estilo_encabezado = ParagraphStyle(
    'EstiloEncabezado', parent=estilos['Normal'],
    fontName='Helvetica-Bold', fontSize=10,
    textColor=colors.whitesmoke, alignment=1
)

# 3. Lista story (carrito de compras)
story = [
    Paragraph("Informe Ejecutivo de Ventas Tecno", estilo_titulo),
    Paragraph(f"<b>Total de Productos Procesados:</b> {len(datos)}", estilo_cuerpo),
    Spacer(1, 12)
]

# 4. Convertir celdas a Paragraph
encabezados_pdf = [Paragraph(f"<b>{h}</b>", estilo_encabezado) for h in encabezados]
filas_pdf = [
    [
        Paragraph(str(r[0]), ParagraphStyle('C', parent=estilo_cuerpo, alignment=1)),
        Paragraph(str(r[1]), ParagraphStyle('L', parent=estilo_cuerpo, alignment=0)),
        Paragraph(str(r[2]), ParagraphStyle('C', parent=estilo_cuerpo, alignment=1)),
        Paragraph(f"${r[3]:,.2f}", ParagraphStyle('R', parent=estilo_cuerpo, alignment=2)),
        Paragraph(str(r[4]), ParagraphStyle('R', parent=estilo_cuerpo, alignment=2))
    ]
    for r in datos
]

# 5. Configurar anchos y tabla
anchos_cols = [40, 160, 100, 80, 80]
tabla_pdf = Table([encabezados_pdf] + filas_pdf, colWidths=anchos_cols, repeatRows=1)

# 6. Aplicar estilo visual con TableStyle
tabla_pdf.setStyle(TableStyle([
    ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#1A365D")),
    ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
    ("GRID", (0, 0), (-1, -1), 0.5, colors.HexColor("#CBD5E0")),
    ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#F7FAFC")]),
]))

story.append(tabla_pdf)

# 7. Compilar PDF
doc.build(story)
```

Paso 5: Código Fuente Completo e Integrado

A continuación, se presenta el programa completo integrado de principio a fin. Deberás escribirlo en tu editor de código para ejecutar el flujo completo:

```python
import openpyxl
from tabulate import tabulate
from reportlab.lib.pagesizes import letter
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib import colors

# 1. Cargar el archivo Excel
wb = openpyxl.load_workbook("ventas_tecnologia_v2.xlsx", data_only=True)
sheet = wb.active

# 2. Leer datos dinámicamente
data = []
for row in sheet.iter_rows(min_row=2, values_only=True):
    data.append(list(row))

# 3. Mostrar en consola con Tabulate
headers = ["ID", "Producto", "Categoría", "Precio ($)", "Ventas (Unid)"]
alignments = ("center", "left", "center", "right", "right")

print(f"\n=== REPORTE DE VENTAS EN CONSOLA ({len(data)} PRODUCTOS) ===")
print(tabulate(data, headers=headers, tablefmt="fancy_grid", colalign=alignments))

# 4. Generar el informe en PDF
doc = SimpleDocTemplate("reporte_ejecutivo_ventas.pdf", pagesize=letter)
styles = getSampleStyleSheet()

title_style = ParagraphStyle(
    'TitleStyle', parent=styles['Title'],
    fontName='Helvetica-Bold', fontSize=18,
    textColor=colors.HexColor("#1A365D"), spaceAfter=10
)
body_style = ParagraphStyle('BodyStyle', parent=styles['Normal'], fontSize=9)
header_style = ParagraphStyle('HeaderStyle', parent=styles['Normal'], fontName='Helvetica-Bold', fontSize=10, textColor=colors.whitesmoke, alignment=1)

story = [
    Paragraph("Informe Ejecutivo de Ventas Tecno", title_style),
    Paragraph(f"<b>Total de Productos Procesados:</b> {len(data)}", body_style),
    Spacer(1, 12)
]

header_paragraphs = [Paragraph(f"<b>{h}</b>", header_style) for h in headers]
data_paragraphs = [
    [
        Paragraph(str(r[0]), ParagraphStyle('C', parent=body_style, alignment=1)),
        Paragraph(str(r[1]), ParagraphStyle('L', parent=body_style, alignment=0)),
        Paragraph(str(r[2]), ParagraphStyle('C', parent=body_style, alignment=1)),
        Paragraph(f"${r[3]:,.2f}", ParagraphStyle('R', parent=body_style, alignment=2)),
        Paragraph(str(r[4]), ParagraphStyle('R', parent=body_style, alignment=2))
    ]
    for r in data
]

col_widths = [40, 160, 100, 80, 80]
pdf_table = Table([header_paragraphs] + data_paragraphs, colWidths=col_widths, repeatRows=1)
pdf_table.setStyle(TableStyle([
    ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#1A365D")),
    ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
    ("GRID", (0, 0), (-1, -1), 0.5, colors.HexColor("#CBD5E0")),
    ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#F7FAFC")]),
]))

story.append(pdf_table)
doc.build(story)
print("\n[ÉXITO] Archivo 'reporte_ejecutivo_ventas.pdf' generado correctamente.")
```

5. Retos Prácticos (Evaluación de Aprendizaje)

Una vez que hayas ejecutado y comprendido el script tutorial, deberás modificar el código para resolver los siguientes ejercicios independientes:

- **Reto 1 — Filtrado por Criterio de Precio:** Modifica la lectura de datos para almacenar y presentar (tanto en la consola con tabulate como en el PDF) únicamente los productos cuyo precio unitario sea superior a $100.
- **Reto 2 — Métricas y Fila de Resumen:** Escribe el código necesario para calcular la suma total de unidades vendidas y el precio promedio de los productos. Muestra estas métricas al final de la consola y agrega una fila final resumida en la tabla del PDF.
- **Reto 3 — Búsqueda Dinámica por ID de Producto:** Solicita al usuario un número de ID mediante la función input(). El programa debe buscar dicho ID en la matriz de datos y mostrar únicamente la fila correspondiente con tabulate. Si el ID no existe, debe mostrar un mensaje de advertencia.
- **Reto 4 (Avanzado) — Menú Interactivo de Consola:** Implementa un bucle while True con un menú interactivo que permita al usuario elegir entre: 1) Ver catálogo completo, 2) Filtrar por precio (> $100), 3) Buscar por ID, 4) Exportar informe PDF y 5) Salir del programa.

6. Criterios de Evaluación y Entregables

Entregables Requeridos:

- Archivo de código fuente de Python nombrado: Taller_Excel_PDF_NombreApellido.py
- Documento PDF generado automáticamente nombrado: reporte_ejecutivo_ventas.pdf

Rúbrica de Evaluación:

Porcentaje	Criterio de Evaluación	Indicador de Logro
60%	Ejecución del Tutorial Base	El script lee ventas_tecnologia_v2.xlsx, muestra la tabla en consola con tabulate y genera el PDF correcto.
25%	Resolución de Retos 1, 2 y 3	Aplica filtros por precio, calcula métricas de resumen e implementa la búsqueda individual por ID.
15%	Menú Interactivo y Código Modular	Estructura un menú con bucle while y organiza las tareas mediante funciones modulares (def).

7. Entregables y Modalidad de Entrega

La entrega es de carácter estrictamente individual. Se debe presentar:

1. Script de Python (.py) con el código fuente resuelto.
2. Reporte en PDF (.pdf) generado automáticamente por el script.
3. Capturas de pantalla como evidencia de la ejecución formateada en consola (tabulate).

Procedimiento de Entrega:

1. Publicación en GitHub: Cargar los archivos requeridos en un repositorio público de GitHub.
2. Registro en el Formulario: Copiar el enlace del repositorio y registrarlo a través del formulario (Form) adjunto en el cuerpo del correo institucional.